Personal Finance Tracker and Analyzer

Student Project Team
Maria Davydova, Maxim Ivanov, Alexey Mikhailenko, Nikolay Tagin
https://github.com/taginnikolaj/Personal-finance-tracker

***Abstract*—This report presents the Personal Finance Tracker and Analyzer project, a tool for automated accounting of personal expenses with support for manual input, downloading data from files, OCR receipt recognition, and visualization of financial statistics. The project is implemented in Python using the libraries pandas, matplotlib, requests and others. The system allows users to analyze their expenses by category, day, and month, as well as track spending against category budgets. The solution is aimed at helping people improve financial discipline and reduce stress associated with unaccounted expenses.**

I. Introduction

The modern rhythm of life often leads to people losing control of their finances. The lack of daily expense tracking leads to overspending, poor budgeting, and financial stress. Existing solutions such as Excel or mobile applications either require manual work or are too complex for beginners. Our project offers a simple, flexible, and automated Python-based solution that can be easily expanded to add AI, API, and new features. The goal is to provide the user with an intuitive interface for analyzing their expenses without having to be a programmer.

II. Problem Statement

People need a simple, accessible way to track, analyze, and understand their personal finances.

The objectives of the project are:

- to provide the possibility of manually entering transactions;
- to support downloading data from CSV and JSON files;
- to implement the deletion of erroneous records;
- to add the receipt recognition function via OCR;
- to provide analysis tools: statistics, categorization, and trends;
- to visualize data in the form of graphs and tables;
- to make the system extensible and convenient for further development.

III. Existing Work

There are various financial accounting applications on the market:

- Excel / Google Sheets require manual entry of formulas and are prone to errors.
- Mobile applications are closed systems with limited functionality and paid subscriptions.
- Python libraries such as finance-tracker and budget-app often do not have a GUI and require programming knowledge.

Our solution combines the advantages of all approaches: automation as in Excel, flexibility as in Python, ease of use as in mobile applications, but free and open.

IV. Objectives and Scope

Objectives:

- Create a working prototype of an expense tracker.
- Implement basic functions: input, download, analysis, and visualization.
- Integrate OCR for automatic data entry from receipts.
- Ensure the readability of the code and the possibility of its extension.

Scope: personal use for budget control.

V. Requirements

A. Functional Requirements

- The ability to manually enter transactions: date, amount, category, and description.
- Downloading data from CSV and JSON files.
- Deleting transactions by ID.
- Receipt recognition via OCR.
- Automatic calculation of statistics: total amount, average, min/max, and quantity.
- Grouping expenses by categories and dates.
- Charting: pie chart, bar chart, line chart, and histogram.
- Displaying results in tables and graphs.
- Category budget management with budget vs actual reporting.

B. Non-functional Requirements

- The code should be readable and well documented.
- The program should work in a standard Python environment without complex installations.
- It should be possible to easily add new functions such as API, ML, and web interface.
- The output interface should be understandable even for non-programmers.
- Error handling is required when entering data and working with the API.

VI. Approach and Design

A. Project Architecture

The project is implemented on a modular basis, which ensures a clear separation of responsibilities between components and simplifies code maintenance.

Main modules:

- The data model is a `Transaction` class that describes a single financial transaction.
- The managing component is the `FinanceManager` class, which is responsible for storing, processing, and analyzing all transactions.
- Configuration settings provide a centralized settings block for controlling program behavior without modifying core code.
- Auxiliary functions provide utilities for searching files, parsing user input, and error handling.
- The OCR module provides integration with an external API for recognizing receipts.
- The visualization module is responsible for construction of graphs based on the processed data.

This architecture allows independently testing and developing each component, as well as easily adding new functions.

B. Technology Selection

- Programming language: Python 3.x.
- Runtime environment: Python environment with notebook support.

Main libraries:

- `pandas` for storing, filtering, and aggregating tabular data.
- `matplotlib` for building static graphs such as pie charts, bar charts, line charts, and histograms.
- `requests` for sending HTTP requests to an external OCR API.
- `csv` and `json` as standard modules for reading and writing data in popular formats.
- `re` for using regular expressions when parsing text from receipts.
- `pathlib` for convenient and cross-platform work with file paths.

All dependencies are standard or easily installed via `pip`, which makes the project portable and reproducible.

C. Class Design

- **Transaction Class.** The class represents a model of a single financial transaction. It encapsulates the transaction data and ensures its validation. The `validate_date()` and `validate_amount()` methods ensure that the date matches the `YYYY-MM-DD` format, and the amount is a positive number. This prevents incorrect data from entering the system. The `to_dict()` method is also implemented, which converts an object into a dictionary, which is convenient for saving in JSON or displaying in a table.
- **FinanceManager Class.** This class is the central control component of the system. It stores a list of all transactions and provides methods for working with them. The class supports loading data from CSV and JSON files, adding transactions manually, deleting by index, as well as calculating key metrics and grouping data for analysis.
- **OCR Module.** An external OCR-Space service is used to automate data entry from paper receipts. The module consists of two main functions:
  - `call_ocr_space(image_path, api_key, language='auto')` sends the receipt image to the OCR-Space server via a POST request and receives a JSON response with the recognized text.
  - `parse_receipt_items(ocr_result)` analyzes the response from the API: it extracts rows with product names and prices, matches them by vertical position, ignores service words such as total, tax, and subtotal, and returns a DataFrame with columns `description`, `amount`, and `category`.

VII. Implementation

- **Setting up the environment and importing libraries.** At the initial stage, the necessary Python libraries were connected to work with data, the file system, and graphics. Special attention is paid to configuring visualization styles and limiting the number of output rows in tables to avoid overloading the interface with unnecessary information.
- **Development of the Transaction class.** The Transaction base class was created, representing a single financial transaction. The key feature of this class is the implementation of strict validation of input data. The `validate_date()` method ensures dates follow the `YYYY-MM-DD` format, while `validate_amount()` guarantees positive numeric values. The `to_dict()` method provides convenient conversion for display and serialization.
- **Development of a management class (FinanceManager).** The central element of the architecture is the FinanceManager class, which aggregates all transactions and provides comprehensive tools for working with them.
- **System settings.** For the convenience of the user, a separate configuration block has been allocated, allowing flexible control of the program without changing the main code.
- **Auxiliary utilities.** A set of auxiliary functions has been developed to automate routine tasks:
  - File search: automatically search for files using preferred names or standard patterns.
  - Input parsing: converts comma-separated text lines into structured transaction data.
  - ID processing: reads and validates a list of transaction identifiers for deletion.
- **Loading and clearing data.** A data processing pipeline is implemented: at startup, the system automatically attempts to load existing files (CSV or JSON). After loading, manual records from the settings are added, and then transactions specified for deletion are removed by ID. All changes are immediately reflected in the current data table with proper indexing.

- **Integration of optical recognition (OCR).** To automate data entry from paper media, a module for working with the external API of the OCR-Space service has been implemented:
  - Sending the request: `call_ocr_space()` sends the receipt image to the processing server with appropriate API key and language settings.
  - Response parsing: `parse_receipt_items()` analyzes the received text using spatial coordinates from the OCR result. The algorithm matches product descriptions, typically on the left side of the receipt, with corresponding prices, typically on the right side, by comparing vertical positions. Service labels such as total, tax, and subtotal are ignored. Dates and other non-item lines are filtered out.
- **Receipt processing.** A complete receipt reading scenario is implemented: if OCR is enabled in the settings, the system finds the image file, sends it for recognition, parses the results, and automatically adds the recognized items to the general database of transactions with the current date or a specified date from settings. Error handling is provided: if recognition fails or the receipt file is not found, the program continues execution without interruption.
- **Final analysis and reporting.** At the final stage, the data is aggregated and displayed in a convenient way:
  - A complete table of all transactions with assigned IDs after applying filters.
  - A block of main statistics: total expenses, average receipt amount, maximum and minimum purchases.
  - Detailed breakdown of expenses by category with percentage of total.
  - Daily, weekly, and monthly totals tables for tracking spending dynamics over different time horizons.
  - Budget report comparing actual spending against user-defined category budgets with status indicators.
- **Data visualization.** A comprehensive unified dashboard is constructed to visually represent the results:
  - Category totals bar chart (horizontal): shows absolute amounts of expenses by category, sorted descending for easy comparison.
  - Pie chart: displays the share of each category in total expenses.
  - Budget vs actual bar chart: compares actual spending against category budgets, with visual indicators for overspending.
  - Daily trend line chart: shows the change in daily expenses over time, allowing identification of spending patterns.
  - Weekly summary bar chart: aggregates expenses by week for broader trend analysis.
  - Monthly summary bar chart: shows spending patterns across months.
  - Transaction amount distribution histogram: illustrates the frequency of transaction amounts, with the mean value highlighted.

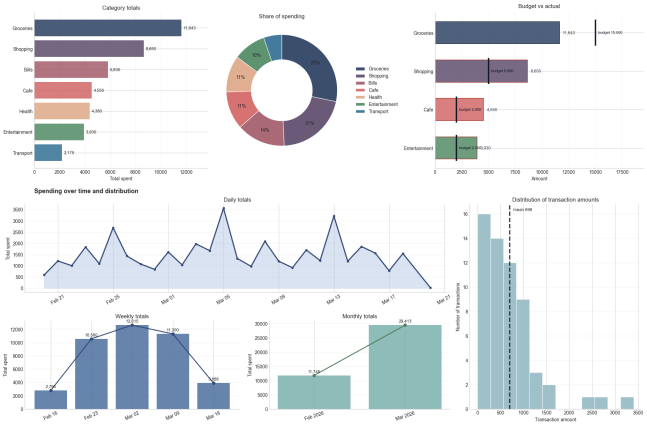

Fig. 1. Finance charts dashboard — visualization output.

- **Month-end forecasting.** Based on the current month's spending pace, the system generates a forecast of month-end expenses. The module calculates:
  - days elapsed in the current month and remaining days;
  - daily spending pace per category and overall;
  - projected total by category and overall at current pace;
  - projected gap compared to category budgets;
  - safe daily spending limit to stay within budget for each category.

  The forecast is displayed in a table with key metrics. Categories likely to exceed the budget are highlighted. A horizontal bar chart visualizes the forecast: full bars represent projected totals, dark bars show amounts already spent, and vertical markers indicate budget limits. This feature helps users proactively adjust their spending to stay on track with financial goals. All charts are organized in a single cohesive dashboard layout with consistent color coding by category, ensuring clear visual communication of financial insights. Chart styling includes appropriate grid lines, labels, and formatting for optimal readability.

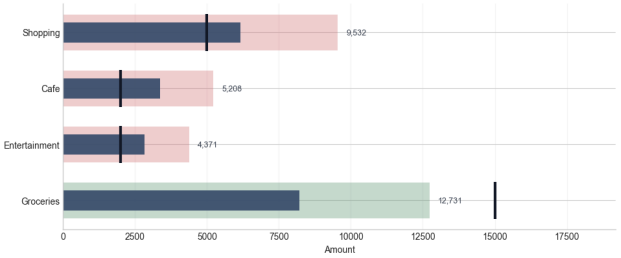

Fig. 2. Month-end forecast — chart.

VIII. Conclusion

During the project, a fully functional Personal Finance Tracker prototype was successfully developed and tested. The solution supports multiple data input sources: manual entry, CSV and JSON files, and automatic receipt recognition via the OCR-Space API. The system provides powerful analytical tools, including financial metrics, category and time-based

grouping, budget tracking, and a comprehensive visualization dashboard. Key technical challenges included error handling for external API interactions, robust parsing of diverse receipt formats, and seamless synchronization between program modules. These were successfully addressed, resulting in a stable application with a clean modular architecture. The project has significant potential for future development: transitioning to a web application with Flask or Django, integrating with banking APIs for automatic transaction imports, applying machine learning for expense categorization and spending prediction, and developing a native mobile application. The created prototype serves as a solid foundation for a comprehensive personal financial management ecosystem.

ACKNOWLEDGEMENT

- Maria Davydova: development of the Transaction class, implementation of download functions from CSV and JSON, preparation of accounting documentation.
- Maxim Ivanov: development of the FinanceManager class, implementation of the budget system and the `budget_report` report, preparation of the presentation.
- Alexey Mikhailenko: integration of the OCR module, receipt processing, and API interaction configuration.
- Nikolay Tagin: data visualization, development of the month-end forecast module, design of final tables and graphs, implementation of the unified charts dashboard with seven chart types.